T7 Half B: obstacle-avoidance planner

Inputs, outputs, every step with its equations, and where it is in the code. Fayrouz Moharam, 1 November 2026. Code: `t7_obstacle_avoidance/` (branch T4_Velocity_Profile, not committed yet). Tests: `tests/obstacle_avoidance/test_planner.py`.

In one sentence: every 0.1 s the planner takes Half A's forbidden zones, lays a grid over the next 8 m of track, finds the cheapest free path past the zones (Liniger's corridor method), turns the free strip around that path into a corridor, bends the racing line inside it (sampling by default, graph as backup), checks it, and hands the MPC its 9 preview stages.

1. The plan at a glance

#	Step	What it does	Where in the code	When
0	Widen the cones (disparity extender idea)	Each cone becomes a zone grown by its size, its position error, half the car and a buffer. The car's centre can then be planned as a point.	Half A: `t7_avoidance/perception_obstacles.py: keep_outs()`; extra margin: `grid.py: EXTRA_WIDEN_M`	Every cycle, before Half B
1	Window	Stations every 0.25 m for the next 8 m, with the racing line, walls and curvature at each.	`grid.py: window()`	Every cycle
2	Corridor grid	Cells every 5 cm across the track; a cell is blocked by a wall margin, a zone, or a side Half A says is closed.	`grid.py: blocked(), blocked_with_sides(), build_grid()`	Every cycle
3	Side lock	A zone less than 1.5 m ahead keeps the side chosen before (no flicker).	`plan.py: _lock_sides()`	Every cycle
4	Cheapest path (DP)	Liniger's shortest path through the free cells: near the racing line, smooth, near last cycle's path. No path = stop before the blockage.	`dp.py: dp_path()`	Every cycle
5	Corridor	At each station, the free strip around the path. The zones are never inside it.	`plan.py: _corridor()`	Every cycle
6	Sides and apexes	Which side each zone is passed on, and the line to hold beside it.	`dp.py: sides()`; Pass list in `plan.py: plan_cycle()`	Every cycle
7	Bent racing line	Graph and sampling both built; the quicker that passes the checks is sent. Must stay in the corridor.	`detour.py: sampling_detour(), graph_detour()`	Every cycle
8	Shape and time	Heading, curvature, fastest speed and time of a line.	`detour.py: frenet_shape(), speed_profile(), traverse_time()`	Inside step 7 and 9
9	Preview for the MPC	The 9 stages: e_y_ref, e_psi_ref, kappa_ref, v_ref, corridor, v_cap.	`plan.py: _cycle_plan(), _stages()`	Every cycle
10	Safety check	Mohamed's 8 runtime checks; a plan that FAILs is never sent.	`t6_feasibility/runtime.py: check_cycle()`, called in `plan_cycle()`	Every cycle
11	Memory	Sides and path kept for the next cycle; cleared on a new perception epoch.	`plan.py: Memory`	End of every cycle

The one call per MPC cycle is `plan, info = plan_cycle(ko, s_abs, e_y, vx, ref, walls, kappa_at, memory=memory)` in `plan.py`. Pass the same `Memory` object every cycle.

2. Inputs

2.1 KeepOuts, from Half A every cycle (`t7_avoidance/types.py`)

Field	Unit	Meaning	Used in
car_s_abs	m	Car position the box was built for	logs

Field	Unit	Meaning	Used in
trust	word	full / frozen / incomplete / stale	logs (Half A already lowered v_cap)
list_epoch	int	Perception's list number; changes after a reset	step 11: clears the side memory
horizon_m	m	Zones listed up to this far ahead (8 m)	step 1: window length
v_cap	m/s	Half A's speed cap	step 9: copied, only ever lowered
zones	list	One KeepOut per cone ahead	step 2

Each KeepOut:

Field	Meaning	Used in
id	Perception's obstacle id, stable while listed	steps 3, 6, 11
s_from, s_to	Along-track extent of the box, continuous s like s_abs [m]	step 2 (box)
e_y_from, e_y_to	Across-track extent the car's centre must avoid, + left [m]	step 2 (box)
s_center, e_y_center, radius, stretch	The circle shape: centre, radius in real metres, corner stretch	step 2 (circle, if SHAPE = "circle")
gap_left, gap_right	Room left for the car's centre beside the zone [m]	logs
closed_left, closed_right	Gap under 5 cm: no way past on that side	step 2: that side blocked to the wall
near_wall, seen_now	Either side closed; seen in the latest scan	not needed

2.2 The car (StateEstimate) and the fixed reference

Input	From	Meaning	Used in
s_abs	StateEstimate.s_abs_m	Car position along the centre line, never wraps [m]	steps 1, 9
e_y	StateEstimate.e_y_m	Car offset from the centre line, + left [m]	steps 4, 7, 10
vx	StateEstimate.vx_mps	Forward speed [m/s]	steps 7, 9
ref	t5_reference.loader.PlanningReference	Racing line e_y_ref, e_psi_ref, kappa_ref, v_ref at any s	steps 1, 9
walls(s)	apex_track track.width_at(s, d_min=0.1)	w_left, w_right: centre line to each wall [m]	step 1
kappa_at(s)	apex_track track.kappa_at	Centre-line curvature [1/m]	steps 1, 8
memory	plan.Memory	Last cycle's sides and path	steps 3, 4, 11

3. Outputs

`plan_cycle()` returns a **CyclePlan** (Mohamed's class in `t6_feasibility/runtime.py`) and an **info** dictionary.

Field (9 values: stages k = 0..8)	Unit	Meaning
s	m	Stage position; s[0] = the car's s_abs; 0.1 s apart at the car's speed
e_y_ref	m	The bent racing line: where to be
e_psi_ref	rad	Its heading relative to the centre line
kappa_ref	1/m	Its curvature
v_ref	m/s	Speed: T5's, lowered where the bend needs it, never above v_cap
e_y_min, e_y_max	m	Corridor for the car's centre (walls and zones excluded)
v_cap	m/s	One cap for the cycle: Half A's, lowered by Half B if the track is shut

info key	Meaning
method	"sampling", "graph (fallback)", ... or "none" (no way past / nothing passed the checks)
path_found	False = the DP found no way past: the plan stops before it
sides	zone id -> "left" / "right"
times	Seconds to drive the 8 m window, per method tried

info key	Meaning
rejected	Method -> why it was refused (left the corridor, or a check_cycle FAIL reason)
start_moved	The car's own cell was blocked; the path starts at the nearest free cell
reason	When method = "none": why

4. The steps, one by one

Step 0. Widen the cones: the disparity extender idea (done by Half A)

The disparity extender (Otterness 2019) grows every obstacle by half the car plus a margin, so the free gap shifts and the car's centre can be planned as a point that never touches the cone. Half A's zones do exactly this, so Half B does not widen again (that would count the car twice and close real gaps).

```
R = r + 2.45 · sigma + 0.15 + 0.10
d = max( 0.1 , min( 1 - kappa(s)·(e - R) , 1 - kappa(s)·(e + R) ) )
s_from, s_to = s_center -/+ R / d e_y_from, e_y_to = e -/+ R
gap_left = (w_left - 0.25) - (e + R) gap_right = (e - R) + (w_right - 0.25)
closed_left = gap_left < 0.05 closed_right = gap_right < 0.05
```

r = cone radius as seen, sigma = its position error (1-sigma), 2.45 = the 95 % circle of a 2-D error, 0.15 = half the car, 0.10 = buffer, d = the corner stretch of s. Extra widening on top: `EXTRA_WIDEN_M` (0 m by default) is added to every side of the zone in step 2.

Where in the code: `t7_avoidance/perception_obstacles.py: keep_outs()` (Half A); `grid.py: EXTRA_WIDEN_M` (Half B).

Step 1. The window ahead

Stations along the track from the car, for 8 m. s never wraps here; the lookups wrap the lap themselves, so a cone past the finish line needs no special code.

```
s_i = s_abs + i · DS , i = 0 .. 32 (DS = 0.25 m, H = 8 m)
lo_i = -w_right(s_i) + 0.25 hi_i = w_left(s_i) - 0.25
e_ref_i, e_psi_ref_i, kappa_ref_i, v_ref_i = ref.at(s_i) kappa_c_i = kappa_at(s_i)
```

Output: a `Window` (s, e_ref, e_psi_ref, kappa_ref, v_ref, kappa_c, lo, hi).

Where in the code: `grid.py: window()`, called first in `plan.py: plan_cycle()`.

Step 2. The corridor grid (Liniger)

Cells across the track every 5 cm. A cell is free when the car's centre may be there. Each station blocks the whole strip [s_i - DS/2, s_i + DS/2] it stands for, so the path can't slip past a zone's corner between two stations.

```
e_j = e_min + j · DE (DE = 0.05 m)
free[i, j] = lo_i <= e_j <= hi_i and no zone blocks (i, j)
box: blocked if s_to >= s_i - DS/2 and s_from <= s_i + DS/2 and e_y_from <= e_j <= e_y_to
circle: ds = (s_near - s_center) / stretch ; blocked if (e_j - e_y_center)^2 + ds^2 <=
radius^2
closed_left: also block every e_j >= the zone's lowest blocked cell (up to the left wall)
closed_right: also block every e_j <= the zone's highest blocked cell
```

Why the closed flags win: a 3 cm gap can still leave one free 5 cm cell, and the DP would squeeze through it. Output: a `Grid` (e, free[33 x N], zone_cells per zone id, without the closed-side strip).

Where in the code: `grid.py: blocked()` (box or circle, switch SHAPE), `blocked_with_sides()`, `build_grid()`.

Step 3. Side lock (no flicker)

If a zone starts less than LOCK_M = 1.5 m ahead (about 0.25 s at 6 m/s) and a side was chosen for it in an earlier cycle, the cells on the other side of it are blocked. Not applied when Half A has closed a side.

```
if s_from - s_abs <= 1.5 and memory.side[id] = left : free[i, e_j < zone] = False
```

Where in the code: `plan.py: _lock_sides()`.

Step 4. The cheapest path (dynamic programming)

One cell per station, from the car to the end of the window. Each step costs (Stahl et al. 2019 cost terms, Liniger's stay-near-the-last-plan term):

```
node(i, j) = DS · [ W_REF · (e_j - e_ref_i)^2 + W_PREV · (e_j - e_prev_i)^2 ]
edge(k) = DS · W_SLOPE · (k · DE / DS)^2 for a move of k cells, |k| <= k_max
k_max = floor( DS · tan(PSI_MAX) / DE ) = 1 (PSI_MAX = 0.35 rad)
J(32, j) = node(32, j) if free, else inf (any free end cell is a goal)
```

```
J(i, j) = node(i, j) + min_k [ edge(k) + J(i+1, j+k) ] if free(i, j), else inf
```

Solved backwards, so a cell with no free way forward (a dead end) gets inf by itself. The path starts at the car's cell (or the nearest free one) and follows the stored best moves. Weights: W_REF = 1, W_SLOPE = 10, W_PREV = 2. W_REF also makes the path pass on the side nearer the racing line, which is Half A's (and ForzaETH's) side rule.

No path. If no cell at station 0 has a finite cost, the track is shut (for example two zones that each close one side, too close to cross between them). The planner finds where the car's reachable cells run out and stops before it, with Half A's stopping formula:

```
distance = s_cut - DS/2 - s_abs - 0.3
v_cap = a · ( -t + sqrt( t^2 + 2 · distance / a ) ) (a = 3.0 m/s^2, t = 0.3 s reaction)
```

Where in the code: `dp.py: dp_path()` (path or `blocked_at`); stop speed `t7_avoidance/speed_policy.py: stop_speed()`, applied in `plan_cycle()`.

Step 5. The corridor

At each station, walk left and right from the path's cell through free cells. The two ends are the corridor. An end that stops at a wall is set to the exact wall band (lo_i or hi_i), not the last cell, so the racing line that runs along the wall margin still fits. An end that stops at a zone is the last free cell, so the zone is never inside the corridor.

```
lo_c_i = lo_i if the run reaches the wall, else e_a  hi_c_i = hi_i if the run reaches the wall, else e_b
```

Where in the code: `plan.py: _corridor()`.

Step 6. Sides and apexes

For each zone, the side is read back from the path at the stations the zone blocks (path above the zone = left). The apex is the line to hold beside the zone: just inside the narrowest corridor edge the zone makes.

```
side = left if path_i > zone cells at i (majority over its stations), else right
apex = max(lo_c over the zone's stations) + 0.01 (left)
apex = min(hi_c over the zone's stations) - 0.01 (right)
```

Each zone becomes a `Pass(zone_id, side, s_a, s_b, apex)`, `s_a` and `s_b` = first and last station it blocks.

Where in the code: `dp.py: sides()`; Pass list built in `plan.py: plan_cycle()`; `detour.py: Pass`.

Step 7. The bent racing line: two ways

Both bend the racing line by an offset delta(s) = e_bent(s) - e_ref(s), inside the same corridor and with the same sides. A line that leaves the corridor is rejected. alpha starts the move earlier at higher speed (ForzaETH):

```
alpha = 1 + min( v / 6.0 , 0.5 ) inside: lo_c_i <= e_ref_i + delta_i <= hi_c_i at every
station
```

7a. Sampling (default) - Werling et al. 2010. For each zone, 27 candidate moves: 3 distances further out than the apex x 3 lengths to get out x 3 lengths to get back. Each move uses the quintic smoothstep, which starts and ends with zero slope and zero curvature:

```
q(t) = 10t^3 - 15t^4 + 6t^5 , t clipped to [0, 1]
bump(s) = (apex + out - e_ref(s)) · min( q((s - (s_a - L_in·alpha)) / (L_in·alpha)) , 1 -
q((s - s_b) / (L_out·alpha)) )
out in {0, 0.05, 0.10} m L_in, L_out in {0.75, 1.25, 2.0} m
start(s) = delta0 · (1 - q((s - s_0) / (1.0·alpha))) (from the car's own offset)
delta = max(positive bumps) + min(negative bumps) + start (same side: furthest wins;
slalom: adds up)
```

Zone by zone (one pass of coordinate descent), every candidate that stays in the corridor is timed (step 8) and the quickest kept.

7b. Graph (backup) - Stahl et al. 2019 / Liniger. The DP path itself, smoothed twice with a 1 m moving average (9 stations), its first point kept at the car, then clipped into the corridor.

```
delta = smooth( smooth( path - e_ref ) ) ; delta_0 = path_0 - e_ref_0 ; delta = clip(e_ref +
delta, lo_c, hi_c) - e_ref
```

A third method, ForzaETH's spline (pin the apex, join the pins with a curve), was tried and removed on 6 November 2026: it was the slowest (1.87 s vs 1.72 s past a cone on closed1) and never the only one that worked.

Where in the code: `detour.py: sampling_detour()` (with `_smoothstep5()`, `_combine()`), `graph_detour()`, `inside()`.

Step 8. Shape and time of a line

Used to time the candidates (step 7) and to give the MPC the bend's heading, curvature and speed (step 9). Frenet relations (the same one T6 checks):

```
e'(s) = de/ds D = 1 - kappa_c · e
theta = atan2(e', D) (heading vs the centre line)
stretch = sqrt(D^2 + e'^2) (metres of path per metre of s)
kappa = (kappa_c + d theta / ds) / stretch
v_i <= min( sqrt(mu·g / |kappa_i|) , v_cap , 6.0 ) (grip, mu = 1.0, g = 9.81)
forward: v_i+1 <= sqrt( v_i^2 + 2 · 2.0 · step_i ) backward: v_i <= sqrt( v_i+1^2 + 2 · 3.0 · step_i
)
time = sum over steps of 2 · step_i / (v_i + v_i+1) , step_i = DS · (stretch_i + stretch_i+1) / 2
```

Where in the code: `detour.py: frenet_shape(), speed_profile(), traverse_time()`.

Step 9. The preview for the MPC

The 9 stages are where the car will be every 0.1 s at its current speed (as control's own preview). Between two stations the corridor takes the stricter edge of both, so a zone can't sneak in between them. The heading and curvature are T5's plus what the bend adds, so on a clear track the plan equals T5 exactly.

```
s_k = s_abs + max(vx, 0.5) · 0.1 · k , k = 0 .. 8
e_y_min_k = max(lo_c_i, lo_c_i+1) e_y_max_k = min(hi_c_i, hi_c_i+1) (s_i <= s_k < s_i+1)
e_y_ref_k = clip( interp(e_ref + delta) , e_y_min_k , e_y_max_k )
e_psi_ref = e_psi_ref_T5 + theta(e_bent) - theta(e_ref)
kappa_ref = kappa_ref_T5 + kappa(e_bent) - kappa(e_ref)
v_ref_k = min( max( min(v_ref_T5, v_bend, v_cap) , 0.5 ) , v_cap )
```

v_bend = step 8's profile on kappa_ref with grip and braking only (the MPC itself decides how to accelerate).

Where in the code: `plan.py: _stages(), _cycle_plan()`.

Step 10. Safety check and choice

Each candidate plan goes through Mohamed's `check_cycle()` before it is timed. A FAIL is never sent. The 8 checks:

#	Check	Fails when
1	No broken numbers	any NaN / inf in the plan or the car's state
2	Same track	StateEstimate track_hash differs from the reference's
3	Starts at the car, moves forward	s[0] differs from s_abs, or s not increasing
4	Corridor open	e_y_max - e_y_min < 5 cm at any stage
5	Line inside its corridor	e_y_ref outside [e_y_min, e_y_max]
6	Car inside its corridor	car more than 10 cm outside (0 to 10 cm = WARN)
7	Steerable	|kappa_ref| > tan(0.40) / 0.30 = 1.41 1/m
8	Speed allowed	v_ref above v_cap, or below 0.5 m/s

Choice: with METHOD = "fastest" (the default) both lines are built and checked every cycle, and the one with the lower time (step 8) that passes is sent; if neither passes, the racing line at 0.5 m/s. With METHOD = "sampling" or "graph" only that one is built (the other one is the backup if it fails).

Where in the code: `t6_feasibility/runtime.py: check_cycle()`; the loop in `plan.py: plan_cycle()`; `plan.py: METHOD`.

Step 11. Memory for the next cycle

Kept: the side of each zone and the DP path (used as e_prev in step 4, Liniger's 'stay near the last plan'). Cleared when list_epoch changes (perception restarted, ids restart). A zone's side is forgotten once the zone is behind the car.

Where in the code: `plan.py: Memory`, updated at the end of `plan_cycle()`.

5. One cycle, in order and in time

Order	Call	Typical time (laptop VM, 1 cone)
1	window() + build_grid() + _lock_sides()	about 0.3 ms
2	dp_path()	about 0.7 ms
3	_corridor() + sides() + Pass list	about 0.3 ms
4	sampling_detour() (27 candidates per zone, each timed) + graph_detour()	about 2.5 ms
5	_cycle_plan() + check_cycle()	about 0.3 ms
	Whole cycle	**4.4 to 5.1 ms mean** with 'fastest' (sampling alone 4.2 to 4.8 ms, graph alone about 2 ms)

The per-call split is an estimate from profiling; the whole-cycle numbers are measured on closed1 (section 6). The MPC runs every 100 ms, so 4 ms leaves it most of the cycle.

6. Which detour is fastest: closed1 and Levine

Perception's two real tracks (apex_track) with their T5 racing lines. A made-up cone (radius 0.075 m, sigma 0.03 m, written in perception's obstacle format) every 2 m around the lap, on the racing line and 0.3 m either side: 72 cases on closed1, 96 on Levine. Half A's `keep_outs()` makes the zone, the car is 3 m before it on the racing line at T5's speed. Run: `python -m t7_obstacle_avoidance.compare` (writes `out/t7_planner_compare/closed1.csv, levine.csv` and a `_summary.txt` for each). Run on the laptop, 6 November 2026.

Track	Method	Worked alone	Mean time past the cone*	Fastest in	Cycle time
closed1	graph	67 / 72	1.732 s	34	2.1 ms
closed1	sampling	71 / 72	1.722 s	33	4.8 ms
closed1	**fastest (default)**	**71 / 72**	**1.666 s**	-	**5.1 ms**
Levine	graph	87 / 96	1.711 s	23	1.9 ms
Levine	sampling	93 / 96	1.634 s	61	4.2 ms
Levine	**fastest (default)**	**96 / 96**	**1.606 s**	-	**4.4 ms**

*Mean over the cases where both worked (67 on closed1, 84 on Levine): time to drive the 8 m window at the car's grip, acceleration and braking limits. Not a lap time, and not yet measured with control's MPC in the loop.

fastest (the default): both lines every cycle, the quicker one that passes is sent. It sent graph on 34 cones and sampling on 37 on closed1, graph on 26 and sampling on 70 on Levine. It is quicker than either one alone (closed1 1.666 s vs 1.722 s, Levine 1.606 s vs 1.634 s) and fails only where both fail: one case, closed1 s 39 with the cone 0.3 m left, in the tightest corner, where the car stays on the racing line at 0.5 m/s. It costs under 1 ms more per cycle than sampling alone, because graph is cheap.

Decision (6 November 2026): METHOD = "fastest". Spline was removed the same day (slowest, never the only one that worked).

7. Tests (`tests/obstacle_avoidance/test_planner.py`, 18 tests, all pass)

Test	Checks
circle_blocks_a_subset_of_the_box	The circle never blocks a cell the box doesn't
closed_side_is_blocked_to_the_wall...	A 3 cm gap Half A calls closed is blocked even if a cell is free
extra_widening_closes_more_cells	EXTRA_WIDEN_M widens the zone
dp_stays_on_the_racing_line_when_clear	No zones: the path is the racing line
dp_finds_no_path_when_two_zones_shut...	Two zones closing opposite sides: no path, cut between them

Test	Checks
clear_track_equals_the_reference	No zones: the plan equals plan_from_reference() exactly
closed_side_passes_the_other_way (x2)	Closed left -> right, closed right -> left; zone never in corridor
both_open_picks_the_side_nearer_the_racing_line	Half A's / ForzaETH's side rule
each_method_goes_round_a_centre_cone (x2)	Graph and sampling each give a non-FAIL plan
fastest_tries_both_and_takes_the_quickest	'fastest' mode picks the minimum time
slalom_two_cones_opposite_sides	Right of the first, left of the second
track_shut_by_two_zones_stops_before_them	No path -> v_cap lowered
zone_past_the_finish_line	Lap wrap with a curved racing line
side_is_kept_once_close...	Side lock: no flip in the last 1.5 m
new_epoch_forgets_the_sides	Perception reset clears the memory
cycle_time	4 zones, both methods: under 100 ms (prints the time)

Run: `python -m pytest tests/obstacle_avoidance`. Whole repo (`python -m pytest tests`): 175 pass, 3 skipped (they need perception's recorded drive run_B). Needs osqp and grad-perception/apex_track on the path.

8. Settings

Setting	Value	File	Meaning
H, DS, DE	8 m, 0.25 m, 0.05 m	grid.py	Window length, station and cell spacing
WALL_MARGIN	0.25 m	grid.py	Car centre to wall (half car 0.15 + buffer 0.10)
EXTRA_WIDEN_M	0 m	grid.py	Extra disparity-style widening on top of Half A's
SHAPE	box	grid.py	box or circle (circle gives about 14 cm more room at zone ends)
W_REF, W_SLOPE, W_PREV	1, 10, 2	dp.py	DP weights: racing line, smoothness, last plan
PSI_MAX	0.35 rad	dp.py	Steepest path against the track (1 cell per station)
METHOD	fastest	plan.py	Both detours, quicker one sent; or "sampling" / "graph" alone
LOCK_M	1.5 m	plan.py	Side kept for zones this close
APEX_MARGIN	0.01 m	detour.py	Apex this far inside the corridor edge
SAMPLE_OUT, SAMPLE_RAMP	0/0.05/0.10; 0.75/1.25/2.0 m	detour.py	Sampling's candidates
RAMP_M, SMOOTH_M	1.0 m, 1.0 m	detour.py	Sampling start ramp; graph smoothing length

9. What is not done yet

• Closed loop with control's MPC on closed1 with cones (C3): the real test of tracking and lap time.

• Replay of Fatma's recorded runs A and B through Half A then Half B (`run_t7_replay.py` runs Half A only today).

• The one tight-corner case on closed1 (s 39) and graph's failures: a curvature limit inside the DP would help.

• Weights and sampling options are starting values (checked on closed1 and Levine with single cones only).

• Pushed to GitHub on 6 November 2026.

10. References

• Liniger, Domahidi, Morari. Optimization-based autonomous racing of 1:43 scale RC cars. Optimal Control Applications and Methods 36(5), 628-647, 2015. arXiv 1711.07300.

• Stahl, Wischnewski, Betz, Lienkamp. Multilayer graph-based trajectory planning for race vehicles in dynamic scenarios. IEEE ITSC 2019, 3149-3154. arXiv 2005.08664.

• Baumann et al. ForzaETH Race Stack: scaled autonomous head-to-head racing on fully commercial off-the-shelf hardware. Journal of Field Robotics, 2024. arXiv 2403.11784.

• Werling, Ziegler, Kammel, Thrun. Optimal trajectory generation for dynamic street scenarios in a Frenet frame. ICRA 2010, 987-993.

• Otterness. The 'Disparity Extender' algorithm, and F1/Tenth. 2019 (cited in Evans et al. 2024, arXiv 2402.18558).